

Plan expérimental – Phase 1

Etude d'un cas simple mono-agent

Sayan Suos – 26 mai 2026

Stage : Étude d'algorithmes d'apprentissage par renforcement pour l'optimisation en temps du transport des bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire.

1 Introduction

Le transport de bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire soulève des problématiques de navigation multi-agents en environnement dynamique. Au sein de notre projet, l'objectif est de minimiser le temps de transport tout en garantissant l'absence de collision, dans un cadre décentralisé où les agents ne disposent que d'observations locales et ne communiquent pas explicitement entre eux. Dans cette optique, un simulateur est actuellement développé afin de modéliser l'environnement étudié et d'expérimenter différentes approches d'apprentissage par renforcement.

Dans un premier temps, nous nous intéressons à un cadre plus général de navigation autonome, en considérant des robots mobiles autonomes (AMR). Cette approche permet d'étudier et de valider les méthodes d'apprentissages dans un environnement plus libre, avant de les adapter progressivement aux contraintes spécifiques du contexte aéroportuaire.

Dans cette première phase expérimentale, nous choisissons d'étudier un algorithme classique d'apprentissage par renforcement profond afin de prendre en main le simulateur et d'implémenter les principaux éléments du problème. Plus précisément, nous entraînons un agent unique dans un environnement comportant des obstacles statiques et dynamiques à l'aide de l'algorithme Soft Actor-Critic (SAC). Ce choix est motivé par le fait que SAC est une méthode off-policy, ce qui permet une meilleure efficacité d'apprentissage. De plus, elle est particulièrement adaptée aux espaces d'actions continues et est largement utilisée dans la littérature pour ses bonnes performances et sa stabilité [1, 2].

2 Modélisation du problème

Le problème est modélisé comme un processus de décision de Markov (MDP), représenté par le tuple (S, A, P, R, γ) où S est l'ensemble des états, A est l'ensemble des actions, $P : S \times S \times A \rightarrow [0, \infty[$ est la probabilité de transition entre les états, $R : S \times A \rightarrow [r_{min}, r_{max}]$ est la fonction de récompense et γ est le facteur de discount.

L'objectif de l'agent est d'atteindre la position cible (x_g, y_g) tout en évitant toute collision, en sélectionnant une action à chaque instant t .

2.1 Espace des états S

L'état de l'agent à l'instant t , noté $\mathbf{s}_t$, regroupe des informations sur l'environnement local et la dynamique du robot :

$$\mathbf{s}_t = [\mathbf{o}_t, \mathbf{p}_t, \mathbf{v}_t, \theta_t].$$

Le terme $\mathbf{o}_t$ correspond à une représentation locale de l'environnement, construite à partir des K dernières observations, sous forme de cartes encodées de taille $W \times H$:

$$\mathbf{o}_t = [o^{t-K-1}, \dots, o^t].$$

Le vecteur $\mathbf{p}_t = [x_g - x_t, y_g - y_t]$ représente la position relative à l'objectif, où (x_t, y_t) est la position courante du robot. Le terme $\mathbf{v}_t = [v_t, \omega_t]$ correspond aux vitesses linéaire et angulaire du robot, et θ_t à l'orientation du robot.

2.2 Espace des actions A

Les actions correspondent aux commandes de contrôle appliquées au robot :

$$\mathbf{a}_t = [v, \omega],$$

avec $v \in [0, v_{\max}]$ et $\omega \in [-\omega_{\max}, \omega_{\max}]$.

2.3 Probabilité de transition P

La dynamique de transition est supposée inconnue et modélise la densité de probabilité du prochain état $\mathbf{s}_{t+1}$, conditionnellement à l'état courant $\mathbf{s}_t$ et l'action $\mathbf{a}_t$.

2.4 Fonction de récompense R

La fonction de récompense est définie comme une combinaison pondérée de plusieurs termes :

$$R = \beta_1 R_g + \beta_2 R_c + \beta_3 R_v + \beta_4 R_s,$$

où R_g récompense la progression vers l'objectif, R_c pénalise les collisions, R_v pénalise les variations de vitesse, et R_s pénalise le non-respect des distances de sécurité. Les coefficients β_i permettent d'ajuster l'importance relative de chaque terme.

Plus particulièrement, la récompense liée à l'atteinte de l'objectif est définie par :

$$R_g = \begin{cases} a & \text{si } \Delta_t^g < 0.05, \\ \Delta_{t-1}^g - \Delta_t^g & \text{sinon,} \end{cases}$$

où Δ_t^g désigne la distance euclidienne entre la position du robot à l'instant t et la position cible, et $a > 0$.

Afin de prendre en compte les contraintes dynamiques du robot, une pénalité est appliquée aux vitesses élevées :

$$R_v = \begin{cases} b|\omega| & \text{si } \omega > \xi^\omega, \\ 0 & \text{sinon,} \end{cases}$$

où $b < 0$.

Enfin, une pénalité importante est appliquée en cas de collisions, ainsi qu'une pénalité plus faible lorsque la distance aux obstacles devient inférieure à un seuil de sécurité.

$$R_c = \begin{cases} c & \text{si collision,} \\ 0 & \text{sinon,} \end{cases} \quad R_s = \begin{cases} d \exp(e(\xi^v - \Delta_t^o)) & \text{si } \Delta_t^o < \xi^v, \\ 0 & \text{sinon,} \end{cases}$$

où Δ_t^o représente la distance au plus proche obstacle, ξ le seuil de sécurité, $c, d < 0$ et $e > 0$.

La construction de cette fonction de récompense s'appuie sur les travaux étudiés [3, 4, 5]. Par ailleurs, l'espace des états est composé de variables de nature différente, elles seront donc normalisées avant d'être utilisées comme entrée des réseaux de neurones.

$$\tilde{x} = \frac{\bar{x} - x}{x_{\text{std}}}, \quad \tilde{y} = \frac{\bar{y} - y}{y_{\text{std}}}, \quad \tilde{\theta} = \frac{\bar{\theta} - \theta}{\theta_{\text{std}}}, \quad \tilde{\Delta}^g = \frac{\bar{\Delta}^g - \Delta^g}{\Delta_{\text{std}}^g} \quad \text{et} \quad \tilde{\Delta}^o = \frac{\bar{\Delta}^o - \Delta^o}{\Delta_{\text{std}}^o}.$$

3 Apprentissage avec l'algorithme Soft Actor-Critic

Une politique $\pi(\mathbf{a}_t | \mathbf{s}_t)$ définit le comportement de l'agent en associant à chaque état une distribution de probabilité sur les actions. Cette politique induit une distribution sur les trajectoires, dont on peut considérer les distributions marginales sur états et les paires actions, notées $\rho_\pi(\mathbf{s}_t)$ et $\rho_\pi(\mathbf{s}_t, \mathbf{a}_t)$ respectivement.

Les algorithmes Actor-Critic reposent sur la combinaison de : (i) un actor network qui prend l'état $\mathbf{s}$ en entrée et génère une action $\mathbf{a}$ en modélisant $\pi_\theta(\mathbf{a} | \mathbf{s})$, et (ii) un critic network qui prend en entrée l'état $\mathbf{s}$ et l'action $\mathbf{a}$ afin d'évaluer la politique en estimant $Q_\phi(\mathbf{s}, \mathbf{a})$.

Dans le cas de SAC, les transitions $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d)$ sont stockées dans un replay buffer $\mathcal{D}$, où $\mathbf{s}' = \rho_\pi(\mathbf{s}, \mathbf{a})$ et $d \in \{0, 1\}$ indique si $\mathbf{s}'$ est terminal ou non.

3.1 Régularisation par entropie

En apprentissage par renforcement, l'objectif classique est de maximiser la somme des récompenses espérées. SAC généralise cet objectif via le principe de maximum d'entropie, afin de favoriser des politiques exploratoires. L'objectif devient alors :

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (R(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))) \right],$$

où $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$ désigne une trajectoire générée par la politique π , et :

$$\mathcal{H}(\pi) = \mathbb{E}_{\mathbf{a} \sim \pi} [-\log \pi(\mathbf{a} | \mathbf{s})],$$

où $\alpha > 0$ est un paramètre de température qui contrôle le compromis entre exploitation et exploration. Il peut être soit fixé avant l'apprentissage, soit optimisé en imposant une entropie cible.

Dans ce cadre, on définit les fonction de valeur associées à une politique π :

$$V^\pi(\mathbf{s}) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(R(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t)) \right) \middle| \mathbf{s}_0 = \mathbf{s} \right]$$

$$Q^\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(\mathbf{s}_t, \mathbf{a}_t) + \alpha \sum_{t=1}^{\infty} \mathcal{H}(\pi(\cdot | \mathbf{s}_t)) \middle| \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right]$$

Ainsi, les fonctions V^π et Q^π sont liées de la manière suivante :

$$V^\pi(\mathbf{s}) = \mathbb{E}_{\mathbf{a} \sim \pi} [Q^\pi(\mathbf{s}, \mathbf{a}) + \alpha \mathcal{H}(\pi(\mathbf{a} | \mathbf{s}))]$$

et l'équation de Bellman de Q^π est :

$$Q^\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\mathbf{s}' \sim P, \mathbf{a}' \sim \pi} [R(\mathbf{s}, \mathbf{a}) + \gamma (Q^\pi(\mathbf{s}', \mathbf{a}') + \alpha \mathcal{H}(\pi(\cdot, \mathbf{s}')))] = \mathbb{E}_{\mathbf{s}' \sim \rho_\pi} [R(\mathbf{s}, \mathbf{a}) + \gamma V^\pi(\mathbf{s}')]]$$

3.2 Apprentissage de la fonction de valeur d'action Q_ϕ

L'équation de Bellman pour la fonction de valeur d'action peut être réécrite comme :

$$Q^\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\mathbf{s}' \sim \rho_\pi, \mathbf{a}' \sim \pi} [R(\mathbf{s}, \mathbf{a}) + \gamma(Q^\pi(\mathbf{s}', \mathbf{a}') - \alpha \log \pi(\cdot | \mathbf{s}'))]$$

Cela signifie que les actions très probables entraînent un petit bonus alors que les actions peu probables entraînent un plus grand bonus, favorisant l'exploration.

Cette équation permet de définir une cible d'apprentissage. L'espérance est approximée par échantillonnage :

$$Q^\pi(\mathbf{s}, \mathbf{a}) \approx r + \gamma(Q^\pi(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi(\tilde{\mathbf{a}}' | \mathbf{s}')), \quad \tilde{\mathbf{a}}' \sim \pi(\cdot | \mathbf{s}'),$$

où r et $\mathbf{s}'$ proviennent du replay buffer, tandis que $\mathbf{a}'$ est échantillonnée à partir de la politique courante.

Pour limiter le biais de surestimation, SAC utilise le double-Q learning : deux fonctions ϕ_1 et ϕ_2 sont apprises, et la plus petite valeur est utilisée. La fonction de perte est :

$$L(\phi_i) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}') \sim \mathcal{D}} \left[(Q_{\phi_i}(\mathbf{s}, \mathbf{a}) - y(r, \mathbf{s}', \mathbf{a}))^2 \right]$$

où la cible est donné par :

$$y(r, \mathbf{s}', \mathbf{a}) = r + \gamma(1 - d) \left(\min_{i=\{1,2\}} Q_{\phi_{i,\text{target}}}(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi(\tilde{\mathbf{a}}' | \mathbf{s}') \right), \quad \tilde{\mathbf{a}}' \sim \pi(\cdot | \mathbf{s}').$$

En effet, pour stabiliser l'apprentissage, chaque réseau est associé à un réseau cible (target network), mis à jour progressivement (soft update) :

$$\phi'_{i,\text{target}} \leftarrow (1 - \lambda)\phi_{i,\text{target}} + \lambda\phi_i,$$

où $\lambda \ll 1$ est très petit.

3.3 Apprentissage de la politique π_θ

La politique est optimisée de manière à maximiser simultanément la valeur et l'entropie. Pour cela, on utilise un reparameterization trick, qui permet d'exprimer une action comme une transformation déterministe d'un bruit aléatoire. Plus particulièrement, on utilise une politique gaussienne suivie d'une transformation par tangente hyperbolique (squashed gaussian policy), afin de contraindre les actions dans un intervalle borné :

$$\tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon) = \tanh(\mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s}) \odot \varepsilon), \quad \varepsilon \sim \mathcal{N}(0, I).$$

De cette manière l'espérance peut être réécrite comme telle :

$$\mathbb{E}_{\mathbf{a} \sim \pi_\theta} [Q^{\pi_\theta}(\mathbf{s}, \mathbf{a}) - \alpha \log \pi_\theta(\mathbf{a} | \mathbf{s})] = \mathbb{E}_{\varepsilon \sim \mathcal{N}} [Q^{\pi_\theta}(\mathbf{s}, \tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon)) - \alpha \log \pi_\theta(\tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon) | \mathbf{s})].$$

L'objectif devient alors :

$$\max_{\theta} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \varepsilon \sim \mathcal{N}} \left[\min_{i=\{1,2\}} Q_{\phi_i}(\mathbf{s}, \tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon)) - \alpha \log \pi_\theta(\tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon) | \mathbf{s}) \right].$$

Le coefficient de régularisation de l'entropie α contrôle explicitement le compromis entre exploitation et exploration. Durant la phase d'évaluation, la politique est rendue déterministe en utilisant l'action moyenne plutôt qu'un échantillon, ce qui conduit généralement à de meilleures performances.

Algorithm 1: Soft Actor-Critic

inputs :

θ : paramètres initiaux de la politique

ϕ_1, ϕ_2 : paramètres initiaux des fonctions de valeurs d'action

$\mathcal{D}$: replay buffer vide

initialisation : $\phi_{1,\text{target}} \leftarrow \phi_1, \phi_{2,\text{target}} \leftarrow \phi_2$

On applique la procédure jusqu'à convergence...

for chaque *itération* **do**

for chaque *étape* dans l'environnement **do**

On observe s puis on exécute a dans l'environnement

$\mathbf{a} \sim \pi_\theta(\cdot | \mathbf{s})$

$\mathbf{s}' \sim \rho_\pi(\mathbf{s}, \mathbf{a})$

$r \leftarrow R(\mathbf{s}, \mathbf{a})$

On stocke les transitions dans le replay buffer

$\mathcal{D} \leftarrow \mathcal{D} \cup (\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d)$

Si s' est terminal, on recommence

if $d = 1$ **then** réinitialiser l'environnement

for chaque *étape* de mise à jour **do**

Échantillonner un batch de transitions aléatoirement

$B \sim \mathcal{D}$

Calculer les targets des fonctions Q

$\tilde{\mathbf{a}}' \sim \pi_\theta(\cdot | \mathbf{s}')$

$y \leftarrow r + \gamma(1 - d) \left(\min_{i=\{1,2\}} Q_{\phi_i,\text{target}}(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi(\tilde{\mathbf{a}}' | \mathbf{s}') \right)$

Mettre à jour Q_{ϕ_1} et Q_{ϕ_2} par descente de gradient, avec :

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d) \in B} (Q_{\phi_i}(\mathbf{s}, \mathbf{a}) - y)^2, \quad \text{for } i = 1, 2$$

Mettre à jour π_θ par montée de gradient, avec :

$$\nabla_{\theta} \frac{1}{|B|} \sum_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d) \in B} \left(\min_{i=\{1,2\}} Q_{\phi_i}(\mathbf{s}, \tilde{\mathbf{a}}_\theta(\mathbf{s})) - \alpha \log \pi_\theta(\tilde{\mathbf{a}}_\theta(\mathbf{s}) | \mathbf{s}) \right), \quad \text{où } \tilde{\mathbf{a}}_\theta(\mathbf{s}) \sim \pi_\theta(\cdot | \mathbf{s})$$

Mettre à jour les target networks :

$\phi_{1,\text{target}} \leftarrow (1 - \lambda)\phi_{1,\text{target}} + \lambda\phi_1$

$\phi_{2,\text{target}} \leftarrow (1 - \lambda)\phi_{2,\text{target}} + \lambda\phi_2$

4 Architecture des réseaux

Pour appliquer l'algorithme SAC, il est nécessaire de définir deux réseaux de neurones : un actor network, qui paramètre la politique π_θ , et un critic network, qui évalue cette politique en estimant la fonction de valeur d'action Q_ϕ . Dans un premier temps, nous utilisons une architecture inspiré

de [6], dans laquelle la politique est apprise à partir de cartes locales égocentriques à l'aide de l'algorithme Distributed Proximal Policy Optimization.

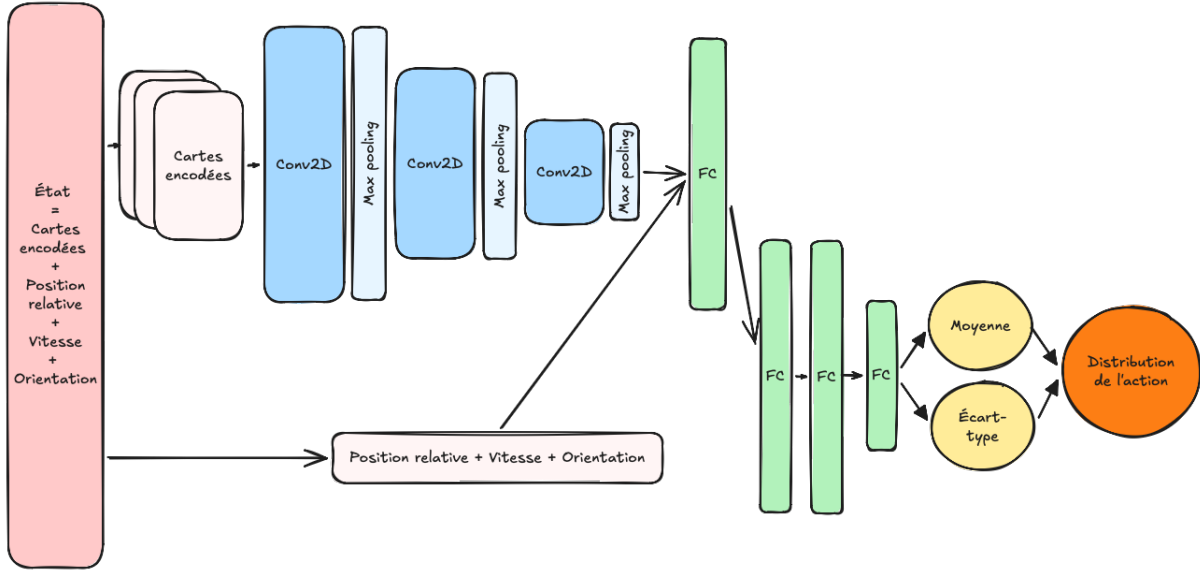


FIGURE 1 – Architecture générale du policy network, inspiré de [6].

Le policy network prend en entrée l'état s_t , composé des dernières cartes encodées, de la position relative à la cible, de la vitesse courante et de l'orientation du robot. Les cartes sont d'abord traitées par un CNN (3 couches Conv2D, 3 couches de max-pooling et 1 couche FC), puis concaténées aux autres composantes de l'état. L'ensemble est ensuite traité par 3 couches FC afin de produire en sortie les paramètres de la distribution de l'action.

Plus précisément, le réseau prédit la vitesse moyenne de l'action $\mu_t = (v_t^{\text{mean}}, \omega_t^{\text{mean}})$, tandis que les écarts-types $\sigma_t = (v_t^{\text{logstd}}, \omega_t^{\text{logstd}})$ sont également appros. Finalement, l'action est échantillonnée selon une loi gaussienne :

$$\mathbf{a}_t \sim \mathcal{N}(\mu_t, \sigma_t),$$

Le critic network repose sur une architecture similaire, à laquelle on ajoute l'action $\mathbf{a}_t$ en entrée. La dernière couche est modifiée afin de produire une estimation de la fonction de valeur d'action $Q_\phi(s_t, \mathbf{a}_t)$.

TABLE 1: Paramètres du modèle

<i>Paramètre</i>	<i>Description</i>	<i>Val.</i>
N	Nombre de cartes encodées utilisées dans l'état	—
H	Hauteur de la carte encodée en m	—
W	Largeur de la carte encodée en m	—
$v_{\min}$	Vitesse linéaire minimale en m/s	0
$v_{\max}$	Vitesse linéaire maximale en m/s	—
$\omega_{\min}$	Vitesse angulaire minimale en rad/s	—
$\omega_{\max}$	Vitesse angulaire maximale en rad/s	—
β_1	Pondération de la progression vers l'objectif dans la fonction de récompense	0.5
β_2	Pondération de l'évitement des collisions dans la fonction de récompense	0.3
β_3	Pondération de la fluidité de navigation dans la fonction de récompense	0.1
β_4	Pondération du respect des distances de sécurité dans la fonction de récompense	0.1
a	Récompense positive liée à l'atteinte de l'objectif	10
b	Récompense négative liée à la vitesse angulaire	-0.1
c	Récompense négative liée à une collision	-10
d	Récompense négative liée aux distances de sécurité	-0.1
e	Pondération de la distance relative au plus proche obstacle	—
ξ^ω	Vitesse de rotation maximale autorisée en m/s	—
ξ^v	Distance de sécurité minimale autorisée en m	—
λ	Coefficient de Polyak	0.005

Références

- [1] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic : Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 10–15 Jul 2018.
- [2] OpenAI. Soft actor-critic, 2023. Spinning Up documentation, consulté le 5 mai 2026.
- [3] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan. Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [4] J. Choi, G. Lee, and C. Lee. Reinforcement learning-based dynamic obstacle avoidance and integration of path planning. *International Journal of Intelligent Robotics and Applications*, 14(5) :663–677, 2021.
- [5] Z. Huang, Z. Ren, T. Chen, S. Cai, and C. Xu. Autonomous navigation and collision avoidance for agv in dynamic environments : An enhanced deep reinforcement learning approach with composite rewards and dynamic update mechanisms. *IET Cyber-Systems and Robotics*, 2024.
- [6] G. Chen, S. Yao, J. Ma, L. Pan, Y. Chen, P. Xu, J. Ji, and X Chen. Distributed non-communicating multi-robot collision avoidance via map-based deep reinforcement learning. *Sensors*, 20(17) :4836, 2020.